

From the Algorithm to the Air

The shared SDR PHY stack & the UnionLabs abstraction layer

SDR platform — OSU experiments deep dive

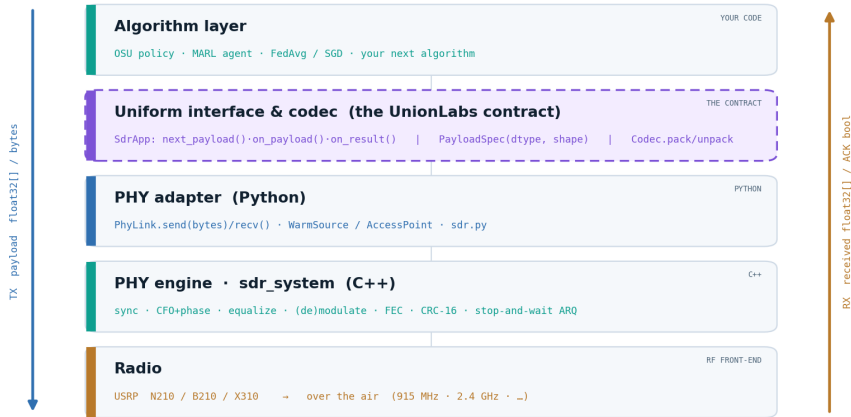
July 26, 2026

The goal: one PHY, many algorithms

- We have **one software-defined-radio PHY** (`sdr_system`, C++) that already sends/receives reliably over USRP hardware.
- We are adding **many algorithms** on top: MARL random access, federated learning, and now **OSU** experiments.
- **Problem:** each algorithm currently talks to the PHY *its own way* with *its own data format*.
- **Deliverable:** a single **abstraction layer** so every algorithm plugs in the same way — declares its output **shape & type**, and the PHY grabs, transports, and returns it identically.

The stack — one payload down, one result up

From the algorithm to the air — the shared SDR PHY stack



The contract (the layer to standardize)

Every algorithm implements the **same three methods** — the PHY *pulls* from it:

- `next_payload()` → `float32[]` — data to transmit (PHY grabs this)
- `on_payload(float32[])` — received data handed back (data apps)
- `on_result(ack: bool)` — delivered / collision (control apps → reward)

And declares **one schema** so shapes never drift:

- `PayloadSpec(dtype, shape)` — e.g. `float32, (n_params,)`
 - `Codec.pack / unpack` — one serializer, self-describing dtype+shape header
- Add an algorithm = implement 3 methods + 1 spec. Nothing below the seam changes.*

Two archetypes, one interface

Data-transfer (FL, MNIST)

- payload **is** the data (a float32 vector)
- uses `next_payload()` + `on_payload()`
- e.g. model / gradient updates

Control / random-access (MARL, OSU)

- payload is **fixed**; the outcome matters
- uses `next_payload()` + `on_result(ack)`
- the **ACK bool = the RL reward**

Both call the PHY through the identical interface.

What the PHY gives every algorithm — for free

- **Modulation:** BPSK · QPSK · 8-PSK · 16/64-QAM · differential (D*)
- **Waveform:** single-carrier (RRC) · **OFDM** (pilots track free-running CFO)
- **FEC:** convolutional+Viterbi · **LDPC** (min-sum) · **turbo** (BCJR); hard/soft; tunable k, iters
- **Reliable delivery:** stop-and-wait ARQ · CRC-16 · ACK over TCP or RF
- **Channel awareness:** energy sensing · `freq_scan.py` · live EVM / pre- & post-FEC BER
- **Multi-node:** N transmitters → 1 access point, per-agent ACK routing (slotted ALOHA)

Worked example — an OSU policy over the radio

- 1 Policy observes its **state** [AoI, queue, channel-busy] (float32) → action: transmit / defer.
- 2 On *transmit*, app hands the PHY a payload via `next_payload()`; adapter fires **one burst** (`--max-attempts 1`).
- 3 PHY engine modulates (DQPSK/SC or QPSK/OFDM), codes it (LDPC/turbo), transmits over the USRP.
- 4 Access point CRC-checks the burst → routes a per-agent ACK back over TCP.
- 5 `on_result(ack)` → **delivered** / **collision** → **reward**; state advances.

This is exactly the MARL loop today — OSU just swaps in its policy.

Code walk (bottom → top):

- ① `sdr_system` (C++) — the PHY engine & `--help` surface
- ② `sdr.py` — auto-generated CLI wrapper
- ③ `marl_phy.py` / `real_channel.py` — WarmSource / AccessPoint (today's adapter)
- ④ `marl_env.py` — obs / action / reward around the ACK
- ⑤ **proposed** `phy_link.py` — the SdrApp + PayloadSpec + Codec contract

Then: port OSU onto the contract as the reference; keep the others unchanged until validated.